

Comprehensive Guide: SpaceX Falcon 9 Landing Predictor

Welcome to the detailed, student-friendly guide for the SpaceX Falcon 9 Landing Predictor project. This document explains every aspect of the project, from its core purpose down to the line-by-line code execution.

1. Project Overview: What and Why?

What is this project? This is a Data Science and Machine Learning web application. Its primary goal is to predict whether a SpaceX Falcon 9 rocket booster will successfully land back on Earth (or on a drone ship) after launching its payload into space.

Why is it useful? Traditionally, rockets were entirely disposable, making spaceflight incredibly expensive. SpaceX revolutionized this by landing and reusing their Falcon 9 first-stage boosters, saving millions of dollars per launch. However, landings are difficult and depend on various mission parameters. By using machine learning on historical data, this project helps estimate the probability of a successful landing based on how heavy the payload is, where it's launching from, and what orbit it's going to. This is crucial for mission risk assessment.

2. Key Components and Their Roles

This project is broken down into a few major components, working together like an assembly line:

- Data Extractor (The Scraper):** Pulls historical launch data directly from Wikipedia. This ensures the model learns from real, up-to-date SpaceX missions.
 - Data Preprocessor (The Cleaner):** Web data is messy. This component cleans the data, fills in blanks, and translates text (like Launch Site names) into mathematical numbers that a machine learning model can process.
 - Machine Learning Model (The Brain):** We use a Random Forest Classifier. It analyzes the cleaned historical data to learn the hidden rules and patterns that lead to a successful landing.
 - Web Dashboard (The Interface):** Built using Dash and Plotly, this is the visual frontend. It provides interactive charts for data analytics and a form for users to input their own mission details to get a live prediction.
-

3. Dataset and Model Evaluation

The Dataset

- **Source:** Wikipedia's list of Falcon 9 launches.
- **Features (The Inputs):** The project isolates three main variables that influence a landing:
 - `Payload mass (kg)`: Heavier payloads require more fuel to launch, leaving less fuel for the landing burn.
 - `orbit`: Different orbits (like LEO vs GTO) require different trajectories and speeds.
 - `Launch site`: Different sites have varying weather patterns and geographic constraints.
- **Target (The Output):** The `Landing_Success` column (1 = Success, 0 = Failure/No attempt).

Model Evaluation

- **Algorithm:** The model used is a **Random Forest Classifier**. Think of it as a "council" of many different decision trees. Each tree makes a prediction, and the forest takes a majority vote. This prevents the model from jumping to conclusions based on single anomalies.
- **Training Strategy:** The data is split 80/20. 80% of the data is used to teach the model, and 20% is hidden away to test it later.
- **Handling Imbalance:** Because SpaceX is very good at landing rockets now, there are many more "Successes" than "Failures" in the dataset. We use `class_weight='balanced'` in the model so it pays equal attention to failures, rather than just blindly guessing "Success" every time.
- **Metrics:** The script calculates the **Accuracy Score**, which represents the percentage of correct predictions made on the hidden 20% test dataset.

4. File Structure Explanation

If you look in the project folder, you will see several files. Here is what they do:

- `train_spacex.py`: The **backend engine**. You run this script to scrape the data, clean it, train the AI model, and save the model to your hard drive.
- `app_spacex.py`: The **frontend user interface**. You run this script to launch the web dashboard in your browser. It loads the saved model to make predictions.
- `spacex_model.joblib`: The **saved brain**. Training a model takes time. Instead of training it every time we open the app, `train_spacex.py` saves the finished model into this file. `app_spacex.py` simply loads it.

- `falcon_9_filtered_data.csv`: A neat, cleaned-up Excel/CSV spreadsheet of the final data used to train the model. It is generated by the training script.
 - `README.md`: The quick-start instruction manual for the project.
-

5. Deep Dive: Line-by-Line Code Explanation

A. `train_spacex.py` (The Training Script)

- **Lines 1-12 (Imports)**: We import necessary toolkits. `pandas` for data manipulation, `requests` for downloading web pages, `sklearn` for machine learning tools, and `joblib` for saving files.
- **Lines 14-30 (Data Scraping)**: We define the Wikipedia URL and send an HTTP request using `requests.get()`. If Wikipedia responds successfully, `pd.read_html()` acts like a magic wand, scanning the web page's HTML and extracting all tables into Pandas DataFrames. We then filter out tables that don't look like flight logs.
- **Lines 31-44 (Fallback & Merging)**: If Wikipedia blocks our automated scraper, the script has a backup plan to generate random dummy data so the program doesn't crash. If the scrape was successful, it merges all the tables into one giant DataFrame.
- **Lines 46-94 (Data Cleaning)**:
 - We filter the `Rocket` column to ensure we only look at 'Falcon 9'.
 - The `Payload mass` comes in as messy text (e.g., "15,000 kg"). We use string replacement and regular expressions (`str.extract(r'(\d+)')`) to extract just the pure numbers.
 - The `Launch site` and `Orbit` columns often have varying acronyms for the same place. We define custom functions (`clean_site`, `clean_orbit`) to standardize them (e.g., combining 'VLEO' and 'LEO' into just 'LEO').
- **Lines 96-106 (Defining the Target)**: We find the column detailing the landing outcome and create a new binary column called `Landing_Success`. If the text contains the word "Success", it gets a 1. Otherwise, it gets a 0.
- **Lines 108-114 (Saving Data)**: The cleaned, pristine dataset is exported to `falcon_9_filtered_data.csv` using `df.to_csv()`.
- **Lines 116-128 (Feature Engineering)**: Machine learning models only understand numbers, not text like "Florida". We isolate our features (`Payload`, `Orbit`, `Site`) and use a `LabelEncoder`. This tool maps text to numbers (e.g., LEO = 0, GTO = 1). We save these encoders in a dictionary so we can use them later in the web app.
- **Lines 130-138 (Model Training)**: `train_test_split` randomly divides our rows into training and testing sets. We initialize the `RandomForestClassifier` and teach it the data using `rf_model.fit(X_train, y_train)`. We then make predictions on the test set and calculate the

accuracy.

- **Lines 139-154 (Saving the Model):** We package the trained model, our text-to-number encoders, and the list of feature names into a dictionary package. We use `joblib.dump()` to save this package as `spacex_model.joblib`.

B. `app_spacex.py` (The Web App Script)

- **Lines 1-5 (Imports):** We import `dash` to build the web framework, `plotly.express` for interactive graphs, and `joblib` to load our saved model.
- **Lines 7-21 (Loading the Model):** We use `joblib.load()` to open `spacex_model.joblib`. We extract the model and the encoders back into active variables. If the file is missing, the app prints an error and stops.
- **Lines 23-74 (Building the UI Layout):**
 - We initialize the web app with `dash.Dash()`.
 - We construct the HTML layout using Dash's HTML components (`html.Div`, `html.H1`).
 - We create two tabs using `dcc.Tabs`.
- **Tab 1 (Launch Analytics):** Contains three `dcc.Graph` components. These use `Plotly` (`px.histogram`, `px.box`) to draw colorful, interactive charts directly from the dataframe saved in our model package.
- **Tab 2 (Custom Prediction Tool):** This is our user form. It contains a numerical input for the Payload (`dcc.Input`) and dropdown menus for Orbit and Site (`dcc.Dropdown`). Notice how the dropdown options are automatically populated using the classes stored in our `label_encoders`! We also add a Predict button (`html.Button`).
- **Lines 76-100 (The Callback / Brain of the UI):**
 - The `@app.callback` decorator is a trigger. It tells the app: "When the Predict button is clicked, take the values from the Payload, Orbit, and Site inputs, and run the `predict_landing()` function."
 - **Data Validation:** The function first checks if the user entered a Payload outside the physical capabilities of the rocket (0 to 25,000 kg). If so, it returns an error: "Prediction can't be determined".
 - **Encoding Input:** The model expects numbers, but the user selected text from the dropdown (like "LEO"). The function uses `label_encoders['Orbit'].transform()` to translate "LEO" back into the exact number the model learned during training.
 - **Prediction:** It packages the three numbers into a `DataFrame` and asks the model to predict the outcome using `rf_model.predict()`.

- **Output:** Depending on whether the model outputs a 1 or 0, the function returns a formatted text string (Success or Failure) back to the web page.
- **Lines 102-103 (Execution):** Starts the local web server so you can view the app in your browser!